

Benchmark d'inférence GPU

Conclusions, correctifs et plan de nouvelle mesure

AMD Radeon AI PRO R9700 • Intel Arc Pro B70 • NVIDIA L40

8 septembre 2026 | Analyse des mesures du 3 au 5 septembre 2026

Conclusion. Les mesures publiées placent la R9700 derrière la B70 et la L40. Elles caractérisent la configuration testée, mais ne démontrent pas les performances maximales de la carte. Des défauts de protocole et des pistes de tuning identifiables justifient une seconde passe avant une recommandation générale.

Débit total à 16 requêtes simultanées

Configuration baseline ; tokens de sortie par seconde. Plus élevé = meilleur.

Modèle / format	AMD R9700	Intel B70	NVIDIA L40
M1 · Qwen3.5-9B / BF16	174,1	344,5	406,6
M2 · GPT-OSS-20B / MXFP4 *	171,6	228,0	270,8
M3 · Qwen3.8-27B / AWQ INT4	88,5	182,4	278,9
M4 · Qwen3.5-35B-A3B / AWQ INT4	272,2	424,8	706,2

Latence de décodage pour un utilisateur

TPOT médiane à C=1, en millisecondes par token. Plus faible = meilleur.

Modèle	AMD R9700	Intel B70	NVIDIA L40
M1	53,8	29,0	22,6
M2 *	72,1	32,2	23,1
M3	33,9	35,7	25,4
M4	71,7	22,9	8,3

* M2 : volume de sortie et réutilisation du cache à revalider ; ses chiffres restent provisoires. AMD devance légèrement Intel sur M3 à C=1, mais NVIDIA reste devant les deux.

Périmètre. Un GPU par test ; 32 Go sur AMD/Intel, 48 Go sur NVIDIA. AMD/Intel : Ryzen 7 9800X3D ; NVIDIA : Xeon Sapphire Rapids. Images vLLM 0.28.0 avec backends ROCm, XPU et CUDA. Charge annoncée : 512 tokens en entrée, 256 en sortie, 50 requêtes, C=1/4/8/16.

Sources : [Rapport AMD](#) · [Rapport Intel](#) · [Rapport NVIDIA](#).

Correctifs du protocole

P0 · Désactiver explicitement le cache de préfixes

Le protocole annonce un cache désactivé, mais le script n’émet pas le drapeau de désactivation. Les logs affichent `enable_prefix_caching=True`. Sur M2/AMD, le taux de réutilisation atteint **76 %**. Le rejeu des prompts au fil des concurrences peut donc favoriser les mesures suivantes.

Correctif : ajouter `--no-enable-prefix-caching` à la commande serveur, puis vérifier la valeur effective dans le log. Avec le générateur actuel de flags YAML, utiliser `no-enable-prefix-caching: true` dans chaque configuration concernée, ou modifier la construction de la commande. Un simple commentaire ou une clé commune non interprétée ne suffit pas.

P0 · Valider les sorties GPT-OSS avant tout classement

À C=16, baseline	AMD	Intel	NVIDIA
Tokens de sortie comptabilisés	2 195	2 121	2 064
Attendus : 50 × 256	12 800	12 800	12 800

Correctif : conserver les réponses détaillées, les raisons d’arrêt et les compteurs usage de l’API. Vérifier les tokens de raisonnement, les arrêts et le comptage client/serveur. La cause n’est pas établie par les fichiers disponibles. Les volumes d’entrée varient aussi légèrement pour M2 selon les runs. Ne pas « corriger » le débit par une règle de trois.

P0 · Chauffer chaque charge et répéter les essais

Deux requêtes de chauffe ne prouvent pas la stabilisation : les logs AMD signalent des compilations Triton pendant l’inférence. Chauffer chaque niveau de concurrence avec la même forme de charge jusqu’à disparition des compilations dans la fenêtre mesurée. Faire au moins **trois passes**, avec redémarrages contrôlés, et augmenter le nombre de requêtes ou la durée pour observer un régime stable.

Rapporter la médiane et la dispersion entre passes. À 50 requêtes, un p99 de latence de requête dépend de très peu d’observations ; ne pas surinterpréter les petits écarts. Les p99 d’intervalles entre tokens portent sur davantage d’échantillons, mais restent sensibles aux compilations et aux transitions.

P1 · Aligner les mesures de puissance et le périmètre

Intel n’a pas de télémétrie de puissance dans ce run : un TDP n’est pas une mesure. Côté AMD/NVIDIA, la collecte entoure le processus client complet, donc inclut préparation et chauffe. Aligner puissance et débit sur la même fenêtre et intégrer les joules. Distinguer énergie GPU et énergie totale du système ; sinon ne pas publier de classement énergétique définitif.

Sources : [construction des commandes et collecte](#) ; [cache M2/AMD](#) ; [JSON M2 AMD](#) ; [Intel](#) ; [NVIDIA](#).

Diagnostic et optimisations AMD

La pile applique déjà des optimisations : image ROCm dédiée, ajustements mémoire, compilation, graphes et kernel W4A16 spécifique RDNA sur M3. Les points suivants ciblent ce qui reste à vérifier.

M4 · Configuration des kernels MoE non ajustée

Fait : le log signale l'absence de la configuration R9700 pour **E=256, N=512, INT4 W4A16** et l'utilisation de paramètres par défaut. Le débit passe de 14,8 tok/s à C=1 à 146,8 à C=4 ; cette rupture mérite investigation.

Action : utiliser l'outil de tuning MoE de la version vLLM installée, en vérifiant la prise en charge du backend WNA16 et des dimensions exactes. Installer la configuration générée, vérifier qu'elle est chargée, puis refaire C=1/4/8/16. Profiler aussi les batchs réels et les compilations. **Gain** : à mesurer.

M3 · Rupture de chemin de calcul et pression du cache

Faits : le débit AMD descend de 60,7 tok/s à C=4 à 57,5 à C=8. Le log confirme **RDNAHybridW4A16LinearKernel**. Le code vLLM consulté bascule entre HIP pour les petits batchs et Triton au-delà d'un seuil de cinq, sous conditions de mémoire locale. À C=16, le cache KV atteint **98,7 %**, avec des requêtes en attente.

Hypothèses à tester : un changement de kernel contribue au décrochage à faible batch ; l'allocation du cache contribue aux attentes à plus forte charge. La concurrence client n'est pas toujours égale à la taille de batch effective. Profiler C=2/3/4/5/6/8/16, puis comparer les budgets mémoire et prefill. Ne pas augmenter arbitrairement le seuil du kernel HIP : il a ses propres limites.

Attention · Comparer le chemin AITER compatible RDNA4

Fait : les logs montrent un repli de l'attention paginée ROCm vers Triton. **Essai** : comparer le backend actuel à **ROCM_AITER_UNIFIED_ATTN**, avec une version AITER compatible installée et **VLLM_ROCM_USE_AITER=1**. Vérifier la sélection effective et la validité numérique. Les kernels AITER disponibles sur RDNA4 diffèrent des chemins CK/ASM d'Instinct : éviter de recopier un profil MI300 complet.

M1 · Crash à 32k et quantification

Fait : le démarrage à 32k échoue sur **HSA_STATUS_ERROR_ILLEGAL_INSTRUCTION**. Ce log ne prouve pas un manque de mémoire. Reproduire le crash avec les versions exactes, puis tester une pile validée pour la R9700. Un essai sans graphes peut aider à isoler le défaut, mais ne constitue pas une optimisation de débit.

Le test « long-context » ne soumet actuellement que des prompts courts. Pour mesurer le contexte long, allonger réellement les entrées. Séparément, tester une version FP8 des **poids** de M1, sous contrôle de qualité ; cela diffère du KV-cache FP8. Vérifier la capacité mémoire avant toute extension aux gros modèles.

Sources : [logs M3](#) ; [logs M4](#) ; [crash M1](#). Références vLLM détaillées en page 5.

Plan de tests proposé

Conserver une référence commune corrigée pour les trois constructeurs. Ajouter ensuite une piste optimisée par constructeur, avec le même modèle et la même qualité cible. Chaque essai ci-dessous revient à cette référence avant de changer un paramètre.

Ordre / essai	Périmètre et modification	Critère de validation
T0 · Référence corrigée Priorité P0	Trois cartes, M1-M4, baseline, C=1/4/8/16. Cache désactivé, chauffe stabilisée, ≥ 3 passes. Diagnostiquer M2 avant de l'agréger.	Tokens et charge comparables ; pas de JIT dans la mesure ; résultats détaillés conservés.
T1 · Attention AITER Priorité P1	AMD, M1 d'abord ; étendre à M3/M4 si compatible. Même modèle et mêmes paramètres, seul le chemin d'attention change.	Backend confirmé dans les logs ; débit et latences améliorés de façon répétable ; qualité vérifiée.
T2 · MoE ajusté Priorité P1	AMD M4, paramètres MoE générés pour la carte et les dimensions exactes. C=1/4/8/16.	Configuration réellement chargée ; évolution du cas C=1 ; gain hors variabilité des passes.
T3 · M3 batch / mémoire Priorité P1	C=2/3/4/5/6/8/16 ; profiler les kernels et batchs. Puis tester séparément les budgets prefill et KV.	Identifier le changement de chemin ; suivre cache, attentes et préemptions ; comparer TPOT et ITL.
T4 · KV FP8 Priorité P2	Après T0, comparer auto et FP8 à prompts courts, puis à contexte réellement plus long, dans les limites mémoire.	Capacité ou débit utiles sous contrainte de latence ; stabilité et qualité. M2 seulement une fois validé.
T5 · Poids FP8 / pile Priorité P2	M1 FP8 séparément ; autre essai avec pile validée R9700. Ne pas changer format et pile à la fois.	VRAM suffisante ; précision acceptable sur corpus réel ; versions et formats documentés.
T6 · Contexte long Priorité P2	Isoler le crash M1 à 32k. Après correction : entrées 4k puis 8k, puis plus longues si capacité validée.	Démarrage stable ; fenêtres et sorties réelles tracées ; latence et capacité mesurées.

Réglages mémoire à explorer sur M3

Autour de la référence existante (utilisation GPU 0,95 ; longueur maximale 2 048 ; budget de tokens batchés 2 048), essayer le budget de prefill 512 / 1 024 / 2 048 lorsque la version et le chunked prefill l'autorisent. Tester une longueur maximale de 1 024 uniquement après vérification des longueurs réelles avec template de chat. Mesurer le cache disponible ; ne pas supposer qu'un réglage libère de la mémoire.

Ne pas changer le contrat pour améliorer le score. Maintenir le nombre de tokens utiles, la qualité, la concurrence et la contrainte de latence. Un résultat obtenu avec un autre format de poids ou un autre moteur doit figurer dans une comparaison distincte.

Livrables attendus et traçabilité

Pour chaque essai

Archiver la commande exacte, la configuration YAML, les variables d’environnement, le digest de l’image, les versions ROCm/vLLM/PyTorch/Triton/AITER, les révisions des modèles et tokenizers, ainsi que les logs serveur et résultats détaillés. Documenter le GPU, son bus PCIe, ses limites de puissance, ses fréquences et sa température ; contrôler l’absence de charge concurrente.

À publier	Présentation attendue
Performance	Débit de sortie et requêtes/s ; TTFT, TPOT et ITL par modèle et concurrence. Médiane et dispersion entre passes.
Fiabilité / capacité	Requêtes réussies et échouées ; tokens réellement comptés ; compilation pendant la mesure ; cache KV, attentes et préemptions.
Qualité	Pour tout changement de précision ou kernel : corpus fixe représentatif, même protocole de scoring, écarts documentés.
Énergie	Puissance mesurée et joules par token sur la même fenêtre que le débit. Distinguer GPU seul et système complet.

Décision proposée. Valider d’abord la référence corrigée. Prioriser ensuite M4 (MoE non ajusté), M3 (batch et cache) et l’attention AITER. Retenir une amélioration seulement si elle est reproductible et respecte les critères de qualité et de latence. Aucun gain chiffré ni rattrapage d’Intel/NVIDIA n’est établi à ce stade.

Références cliquables

- 1. [Dépôt audité, commit 598531350b1726657970fdb46200e96e28d3cea](#)
- 2. [Configuration des modèles et du workload](#)
- 3. [Orchestrateur : flags, chauffe et télémétrie](#)
- 4. [Résultats bruts AMD](#)
- 5. [Résultats bruts Intel](#)
- 6. [Résultats bruts NVIDIA](#)
- 7. [vLLM 0.28.0 : kernel hybride RDNA W4A16 et conditions de bascule](#)
- 8. [vLLM 0.28.0 : activation AITER spécifique RDNA4](#)
- 9. [vLLM 0.28.0 : outil de benchmark et tuning MoE](#)

Limites de cette note. Revue documentaire des scripts, JSON et logs du commit ci-dessus, réalisée à partir de l’analyse du 6 septembre et mise en forme le 8 septembre 2026. Aucun benchmark matériel n’a été réexécuté. Les sources vLLM décrivent le tag public : vérifier leur correspondance avec le code exact des images installées. Les causes non démontrées sont présentées comme des hypothèses ; les essais proposés ne sont pas des gains acquis.